

Banco de Dados II: Transações

■ TRANSAÇÕES

Dados Relacionais II : Transações

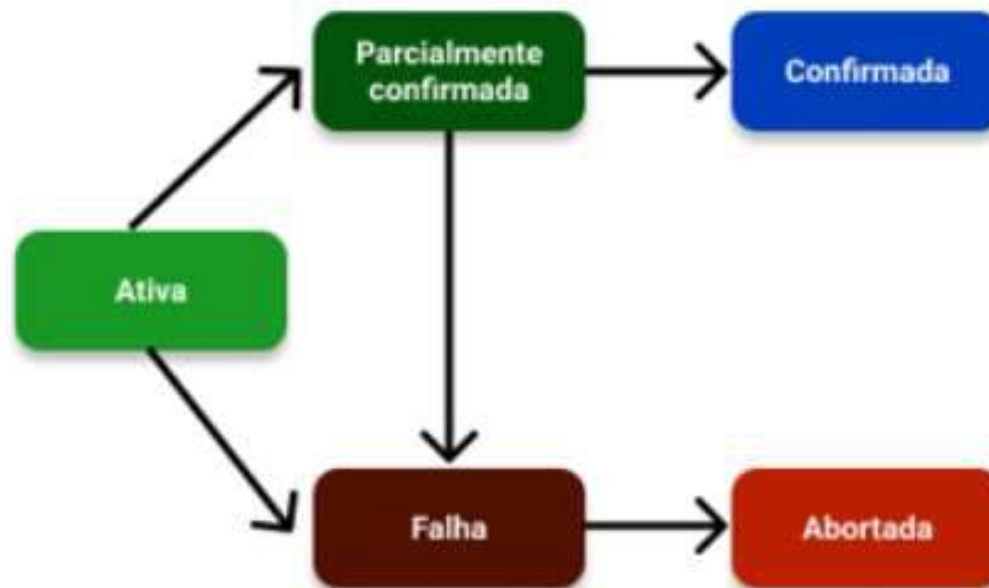
- Sistemas Monousuários x Multiusuários
 - Critério de classificação quanto ao número de usuários que podem usar o sistemas **Concorrentemente**, ou seja, ao mesmo tempo;
 - SGBD Monousuário – um usuário de cada vez pode usar o sistema;
 - SGBD Multiusuário – muitos usuários podem usar o sistema, acessando o banco de dados concorrentemente. Este é o tipo mais largamente utilizado hoje;
 - Pelo fato de múltiplos usuários acessarem os dados ao mesmo tempo, exigem mecanismos sofisticados para gerenciar a **concorrência** e garantir a integridade dos dados.

Dados Relacionais II : Transações

- **TRANSAÇÃO**: É uma *sequencia de operações executada em um banco de dados* que acessa e eventualmente modifica dados;
 - Uma transação começa com o banco de dados consistente, porém, **DURANTE** a execução da transação o banco de dados pode ficar temporariamente inconsistente;
 - Quando a transação é finalizada o Banco de Dados deve ficar consistente;
 - Problemas a considerar no tratamento de transações:
 - Falhas
 - Execução concorrente de varias transações

Dados Relacionais II : Transações

- **TRANSAÇÃO**: É uma *sequencia de operações executada em um banco de dados* que acessa e eventualmente modifica dados;



Estados de uma transação.

Dados Relacionais II : Transações

- **TRANSAÇÃO**: Transações devem obedecer às propriedades **ACID**:
 - **Atomicidade**: A propriedade de atomicidade garante que as transações sejam atômicas (indivisíveis). A transação será executada totalmente ou não será executada.
 - **Consistência**: A propriedade de consistência garante que o banco de dados passará de uma forma consistente para outra forma consistente.
 - **Isolamento**: A propriedade de isolamento garante que a transação não sofrerá interferência de nenhuma outra transação concorrente.
 - **Durabilidade**: A propriedade de durabilidade garante que o que foi salvo, não será mais perdido.

Dados Relacionais II : Transações

- Existem vários casos em que é necessário definir um **BLOCO de instruções (varias transações)**, onde o bloco todo deve ser executado com sucesso, ou nenhum dos comandos dos blocos.
 - Exemplo: Transferir um valor de uma conta para outras

```
alter proc transferencia
(
    @idbancoorigem    int,
    @idbancodestino  int,
    @valor            numeric(18,2)
) as
update banco set ban_saldo = ban_saldo - @valor
where idbanco=@idbancoorigem

Update banco set ban_saldo = ban_saldo + @valor
where idbanco=@idbancodestino

return
```

- E se desse problema no segundo Update??

Dados Relacionais II : Transações

- Para garantir que um bloco seja TODO executado com sucesso, ou nenhum comando do bloco seja executado, o SQL permite estabelecer explicitamente declarações de **início de transação** e **fim de transação**;
- Isso é feito através dos comandos:
 - BEGIN TRAN : Define o início de uma transação.
 - Commit : Define o fim de uma transação
 - Rollback : Cancela uma transação

Dados Relacionais II : Transações

- Ao utilizar BEGIN TRAN antes de um bloco, as alterações feitas no banco somente serão gravadas quando for efetuado um COMMIT.

```
alter proc transferencia
(
    @idbancoorigem    int,
    @idbancodestino   int,
    @valor             numeric(18,2)
) as
Begin tran
    update banco set ban_saldo = ban_saldo - @valor
    where idbanco=@idbancoorigem

    Update banco set ban_saldo = ban_saldo + @valor
    where idbanco=@idbancoDestino

    if @@error>0
        rollback
    else
        commit
return
```

Dados Relacionais II : Transações

■ Exemplo

Begin Tran

Delete pagar

Select * from pagar

Obs: ao executar os comandos acima você verá que TODOS os registros da tabela pagar foram excluídos.

Agora digite **Rollback** e execute somente este comando.

Você verá que os registro da tabela PAGAR REAPARECERAM

BD II: Tratamento de erros

- **Tratamento de erros**

BD II: Tratamento de erros

- **ROLLBACK TRANSACTION:** Em transações, o uso de ROLLBACK é essencial para garantir que, em caso de falha, as operações sejam revertidas para manter a consistência dos dados.
 - Em conjunto com BEGIN TRANSACTION, é possível iniciar uma transação, e caso ocorra um erro durante a execução, a transação é revertida com o ROLLBACK.
- **Variável de erro (@@ERROR em SQL Server):** o SQL Server, possui variáveis de erro, como @@ERROR, que armazena o código do erro ocorrido na última instrução.
 - Esse método pode ser usado para verificar o sucesso de uma operação e, em caso de erro, executar procedimentos de tratamento específicos.

BD II: Tratamento de erros

- **CUIDADO: Nem todos os erros disparam o @@Error.**
Geralmente, erros mais graves, não relacionados com comandos sql não disparam.
 - Erros do Sistema operacional
 - Erros de Interrupção de Sessão
 - Erros de Estouro de Stack / timeout
- O @@ERROR no SQL Server é atualizado **após cada comando executado**, portanto, ele precisa ser verificado **imediatamente** após a execução de cada instrução SQL, pois qualquer comando subsequente redefine o valor de @@ERROR

BD II: Tratamento de erros

```
alter proc transferencia
(
    @idbancoorigem    int,
    @idbancodestino   int,
    @valor             numeric(18,2)
) as
Begin tran
    update banco set ban_saldo = ban_saldo - @valor
    where idbanco=@idbancoorigem

    Update banco set ban_saldo = ban_saldo + @valor
    where idbanco=@idbancodestino

    if @@error>0
        rollback
    else
        commit
return
```

BD II: Tratamento de erros

- **TRY...CATCH:** Muitos SGBDs, como SQL Server, oferecem a estrutura TRY...CATCH para capturar e manipular erros. Dentro
 - O bloco TRY é executado, e se ocorrer um erro, a execução passa para o bloco CATCH, onde é possível tratar o erro, registrar logs, ou executar comandos compensatórios.

BEGIN TRY

instrução

instrução

instrução.....

END TRY

Begin CATCH

instrução

instrução

instrução.....

END CATCH

BD II: Tratamento de erros

```
BEGIN TRY    --- inicia o bloco try
    UPDATE banco
    SET ban_saldo = ban_saldo - @valor
    WHERE idbanco = @idbancoorigem;

    UPDATE banco
    SET ban_saldo = ban_saldo + @valor
    WHERE idbanco = @idbancodestino;
end try
Begin catch
    --- se acontecer qualquer erro no bloco ty,
    --- o fluxo passa para o catch
    select 'Ops, deu erro!. O erro foi ' + ERROR_MESSAGE()
end catch
```

BD II: Tratamento de erros

- **THROW:** É usado para gerar um erro personalizado e interromper a execução de um bloco de código.

THROW Número do erro, 'Mensagem de erro', Estado

- **Número do erro:** O código de erro que você deseja lançar (precisa ser maior que 50000).
- **Mensagem de erro:** O texto que será exibido.
- **Estado:** Estado do erro (normalmente 1).

BD II: Segurança

■ Segurança

BD II: Segurança

- Segurança em banco de dados é uma área bastante ampla que se refere a muitas questões, incluindo:
 - **Questões legais e éticas:** Trata dos direitos de acesso a informações específicas, garantindo que dados pessoais, financeiros e sensíveis sejam acessados apenas por aqueles que possuem autorização legal e ética para tal. Isso envolve conformidade com regulamentações como a **LGPD** (Lei Geral de Proteção de Dados).
 - Questões políticas nos níveis governamental, institucional ou corporativo (quais os tipos de informação que não devem ser tornadas disponíveis publicamente);
 - As necessidades em algumas organizações de identificar múltiplos níveis de segurança e em categorizar os dados e usuários com base nessas classificações. Exemplo: altamente secreto, secreto, confidencial e não confidencial;

BD II: Segurança

- **Confidencialidade, Integridade, Disponibilidade** são os principais pilares da segurança em bancos de dados e são amplamente conhecidos como o modelo **CIA** (Confidentiality, Integrity, Availability).
 - **Confidencialidade:** refere-se à proteção dos dados contra a divulgação não autorizada
 - **Integridade:** a informação deve ser protegida contra modificações impróprias (intencionais ou acidentais);
 - **Disponibilidade:** o acesso à informação deve estar disponível para um usuário ou programa que tenha direito legítimo a esta informação;

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Confidencialidade**

- **Criptografia de Dados:** **Criptografar** dados armazenados e em trânsito para impedir que informações sensíveis sejam acessadas sem autorização.
- **Controle de Acesso Baseado em Funções:** Definir e aplicar políticas de acesso que permitam apenas o nível mínimo de acesso necessário para cada usuário.
- **Autenticação Forte:** Adote autenticação multifatorial e políticas rigorosas de senha para prevenir acessos não autorizados.

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Confidencialidade**

● Criptografia a **nível de BANCO**.

- A criptografia a nível de banco criptografa os arquivos de dados e os arquivos de log do banco de dados, tornando os dados ilegíveis se os arquivos forem acessados fora do SQL Server (ou de outro servidor)
- A criptografia é gerenciada por uma **chave de criptografia do banco de dados (DMK)**, que é armazenada no próprio banco de dados. A DMK é protegida por um certificado armazenado no servidor
- O **TDE - Transparent Data Encryption** é chamado de "transparente" porque **não altera a forma como consultas e operações** são executadas no SQL Server. Os dados são descriptografados automaticamente em tempo real quando acessados por usuários e aplicativos autorizados, e criptografados ao serem salvos em disco, sem que o usuário precise fazer nada..

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

- Proteção da **Confidencialidade**

- Criptografia a **nível de BANCO**.

- Criar uma chave mestra e um certificado no banco master(DMK).
 - Criar uma chave de criptografia no banco de dados específico. (protegida pela DMK)
 - Ativar a criptografia **TRANSPARENTE** no banco de dados.

BD II: Segurança

--1 passo

```
Create master key encryption by password = '12345'  
go
```

- Cria uma chave simétrica chamada **##MS_DatabaseMasterKey##** dentro do banco atual (DMK).
- Essa chave é usada como **base da hierarquia de criptografia local** do banco
- Ela é criptografada de duas formas:
 - Pela **Service Master Key (SMK)** (chave mestra da instância).
 - E pela **senha** que você definiu
- **É importante fazer backup de sua chave**

10:54:20

```
BACKUP MASTER KEY TO FILE = 'C:\backup\meu_banco_dmk.key'  
ENCRYPTION BY PASSWORD = 'SenhaBackup123!';
```

BD II: Segurança

```
--2 passo: criar um certificado  
create certificate certificado_uvv with subject = 'certificado'  
  
SELECT * FROM sys.certificates;
```

CertificadoTDE: é o certificado criada diretamente no SQL Server. Esse certificado será armazenado internamente no banco de dados master

BD II: Segurança

```
USE uvv;
```

```
GO
```

```
-- Criar a chave de criptografia do banco e associá-la ao cert:
```

```
CREATE DATABASE ENCRYPTION KEY
```

```
WITH ALGORITHM = AES_256
```

```
ENCRYPTION BY SERVER CERTIFICATE certificado_aula;
```

```
GO
```

```
-- Depois, basta ativar a criptografia para o banco de dados.
```

```
ALTER DATABASE UVV
```

```
SET ENCRYPTION ON;
```

```
GO
```

Infelizmente, não funciona na versão gratuita (Express)!!!

BD II: Segurança

- Como o próprio nome diz, esta criptografia é **TRANSPARENTE**, ou seja, o usuário não sabe que os dados são criptografados/decriptografados
 - Uma consulta do tipo `Select * from pagar` irá mostrar os dados normais
 - Da mesma forma, um update/insert é feito normalmente
- O sql server criptografa os dados antes de salvar no disco e decriptografa antes de mostrar ao usuário
- O foco é a proteção dos dados em disco, ou seja, contra o roubo do arquivo de dados ou do backup. O **TDE não protege dados de acesso indevido**
- **O backup de um banco criptografado, estará automaticamente criptografado também.**

BD II: Segurança

Tipo de ameaça	Protegido pelo TDE?	Observação
Roubo de arquivos MDF/LDF	✓	Dados criptografados em disco
Roubo de backups (.bak)	✓	Backup também cifrado
SELECT indevido por usuário SQL	✗	Dados descriptografados na consulta
DBA lendo tabelas confidenciais	✗	Tem acesso normal
Ataque via rede / interceptação	✗	Use SSL/TLS para proteger o tráfego
Dados em RAM	✗	São descriptografados em memória

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

- Proteção da **Confidencialidade**

- Criptografia a **nível de COLUNA**.

- `CREATE SYMMETRIC KEY SK_DadosSensíveis
WITH ALGORITHM = AES_256
ENCRYPTION BY CERTIFICATE Certificado_uvv`

A Chave esta sendo protegida pelo certificado

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

- Para fazer um update criptografado

--- Abrir a chave de criptografia usando o certificado

```
OPEN SYMMETRIC KEY SK_DadosSensiveis
```

```
    DECRYPTION BY CERTIFICATE Certificado_uvv;
```

```
update empresa set
```

```
cpf_crypto=ENCRYPTBYKEY(KEY_GUID('SK_DadosSensiveis'), '58601015034') where idempresa=2
```

```
SELECT CONVERT(varchar(50),
```

```
DECRYPTBYKEY(cpf_crypto)) AS cpf_em_texto
```

```
FROM dbo.Empresa WHERE IdEmpresa = 2;
```

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

IMPORTANTE:

- DecryptByKey(cpf_crypto) sempre retorna VARBINARY, logo, é necessário converter para string
- EncryptByKey sempre espera um valor STRING ou binary. Se você quiser criptografar um valor numérico, primeiro você deve converter este valor para string:

```
update empresa set  
valor_crypto=ENCRYPTBYKEY(KEY_GUID('SK_DadosSensitive'  
is'), cast(100.00 as varchar(10)))  
where idempresa=2
```

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Integridade**

- Controle de Validação de Entrada: Validar as entradas para **evitar injeções SQL** e garantir que os dados estejam no formato esperado antes de serem armazenados.
- Controle de Acesso Granular: Implementar permissões adequadas para que apenas usuários autorizados possam modificar dados, reduzindo o risco de alterações acidentais ou maliciosas.
- Uso de Triggers e Constraints: Utilizar restrições de integridade e triggers para evitar modificações indevidas nos dados, assegurando que informações incorretas não sejam inseridas ou alteradas.
- Auditorias e Logs de Atividade: Monitorar todas as atividades no banco de dados, registrando alterações e acessos para detectar e investigar rapidamente alterações inadequadas ou suspeitas.

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Disponibilidade**

- **Backup e Recuperação de Dados:** Realize backups regulares e mantenha um plano de recuperação de desastres para restaurar rapidamente o acesso aos dados em caso de falhas ou ataques.
- **Defesas Contra Ransomware(*) e DoS(*):** Implemente soluções de monitoramento e detecção de ameaças para identificar atividades suspeitas, e use sistemas de defesa contra ataques de negação de serviço (DoS).
- **Redundância e Alta Disponibilidade:** Configure réplicas e failover automático para garantir que o banco de dados permaneça acessível, mesmo em caso de falhas no sistema ou manutenção.
- **Controle de Acesso à Rede:** Restrinja o acesso à rede para impedir acessos externos não autorizados, limitando a exposição do banco de dados.

BD II: Segurança

Como garantir os pilares básicos de segurança (CIA)?

■ Proteção da **Disponibilidade**

- **Ransomware** : é um tipo de malware que criptografa dados e exige um resgate (geralmente em criptomoeda) para restaurar o acesso.
- **DoS (Denial of Service)** é um ataque que visa sobrecarregar um sistema ou servidor, tornando-o incapaz de responder a requisições legítimas. No SQL Server, um ataque DoS pode ocorrer de várias formas:
 - **Sobrecarga de Requisições**: Um invasor pode enviar um grande número de consultas SQL complexas ou pesadas para sobrecarregar o servidor, exaurindo a CPU, a memória e o armazenamento temporário.
 - **Bloqueio de Sessões**: O invasor pode abrir várias sessões no SQL Server e mantê-las ativas, impedindo que usuários legítimos possam acessar o banco.

BD II: Segurança

- **DBA** (Administrador de Banco de Dados): É a autoridade máxima no gerenciamento do sistema de banco de dados, responsável por definir e manter a política de segurança e controle de acesso.
 - O DBA possui uma conta especial com permissões para realizar ações administrativas avançadas, como criação de contas e atribuição de privilégios.
 - **sa** no Sql server , **SYS** no oracle , **root** no mySql
- Os privilégios do DBA incluem comandos para conceder e revogar direitos para contas individuais de usuários ou de grupos de usuários, e comandos para a realização das seguintes ações do tipo:
 - Criação de Contas;
 - Concessão de privilégio discricionários;
 - Revogação de privilégio;
 - Atribuição de nível de segurança;

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
 - **Login de Usuário:** é a identidade no nível do servidor e representa um usuário que possui permissões para se conectar ao **servidor SQL**.
 - O login permite que o usuário se **autentique no servidor**, mas **não concede** automaticamente permissões dentro de um **banco de dados específico**.
 - **Autenticação do Windows:** Utiliza contas ou grupos de usuários do Windows para login.
 - **Autenticação do SQL Server:** Cria logins com credenciais específicas para o SQL Server, independentes do sistema operacional.

BD II: Segurança

■ Criação de Contas: O sql server possui vários “níveis” de autenticação.

- **Autenticação do Windows:**

`CREATE LOGIN [Dominio\lucas] FROM WINDOWS;`

* Quando não existir domínio, deve ser informado o nome do PC

- **Autenticação do SQL Server:**

`CREATE LOGIN lucas with password = '123'`

Importante: O ‘login’ fornece **acesso somente ao servidor de banco de dados**. Os bancos de dados ainda estão inacessíveis.

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
 - Para excluir uma conta de login, basta usar o comando drop:
 - `drop login jlucas` - para uma conta com autenticação sql)
 - `drop login [srv01\jlucas]` - para uma conta com autenticação windows
 - **CUIDADO!!! Para excluir uma conta de login, é necessário considerar algumas regras:**
 - O login excluído não pode estar **logado** (com conexões ativas);
 - O login não pode ter **usuário vinculado em algum banco de dados**
 - O login não pode ser dono de um ou mais objetos (banco, índices, schemas...)

BD II: Segurança

- Os **logins** de um servidor podem ser consultados na tabela **sys.server_principals**

SELECT

```
sid , --Security ID, representa um id único, em hexa  
principal_id, --ID único DENTRO de um BD, não é GLOBAL  
name AS LoginName,  
type_desc AS Tipo,  
create_date AS CriadoEm,  
is_disabled AS Desativado,  
type
```

FROM **sys.server_principals**

-- S = SQL login, U = Windows user, G = Windows group

ORDER BY name;

BD II: Segurança

- **Conta de Usuário:** O usuário do banco de dados é uma entidade de segurança criada dentro de um banco de dados **específico**.
 - Um **'USUÁRIO'** **NÃO** representa uma pessoa, e sim **OQUE** um determinado login pode fazer
 - No SQL Server, um “usuário” do banco de dados geralmente é associado a um **login de servidor**.
 - O **login** autentica o usuário no servidor SQL,
 - O “**usuário**” define o que o usuário pode fazer dentro do banco específico.
 - Usuário de **bancos diferentes** podem estar relacionados ao mesmo login. No **mesmo banco** isso não é possível

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
- **Conta de Usuário:** O usuário do banco de dados é uma entidade de segurança criada dentro de um banco de dados **específico**.

- **CREATE USER** usu_todos **FOR LOGIN** lucas
- **CREATE USER** user_consulta **FOR LOGIN** maria

Para alterar:

- **Alter USER** user_gravacao **with LOGIN** = jose;

Após a criação, deve-se definir oque cada conta de usuário poderá acessar. Pode-se usar o comando **GRANT** ou grupo de regras (Roles)

BD II: Segurança

- Para consultar todas as contas de usuários, basta ver a tabela `sys.database_principals`:

SELECT

```
    sid , --Security ID, representa um id único, em hexa  
    principal_id, --ID único DENTRO de um BD, não é GLOBAL  
    name AS UserName,  
    type_desc AS Tipo,  
    type,  
    authentication_type_desc AS Autenticacao,  
    create_date AS CriadoEm
```

FROM `sys.database_principals`

WHERE type IN ('S', 'U', 'G') -- Filtra usuários comuns

ORDER BY name;

BD II: Segurança

- Você já criou uma **conta de login** no servidor.
- Você criou uma **conta de usuário** e relacionou a mesma a um determinado login;
- Agora, acesse uma tabela qualquer do banco. Oque aconteceu??

BD II: Segurança

- **Você ainda não tem acesso** a nenhum objeto do banco!!! Para ter acesso, você deve definir as permissões da conta, usando **GRANT**, **DENY** e **REVOKE**.
- **GRANT** : Concede permissão a determinado objeto ou coluna de um banco de dados.
 - `GRANT SELECT ON dbo.Empresa TO lucas;`
 - o usuário lucas pode dar `SELECT` na tabela `dbo.Empresa`
 - `GRANT SELECT (Nome, CPF) ON dbo.empresa TO lucas;`
 - o usuário lucas pode dar `SELECT` somente nas colunas `nome` e `cpf` da tabela `dbo.Empresa`
 - `GRANT SELECT ON dbo.Clientes TO lucas WITH GRANT OPTION;`
 - O usuário lucas poder “repassar” a permissão dele para outro usuário

BD II: Segurança

- **Você ainda não tem acesso** a nenhum objeto do banco!!! Para ter acesso, você deve definir as permissões da conta, usando **GRANT**, **DENY** e **REVOKE**.
- **DENY** : Nega explicitamente uma permissão. É o “não pode”, mesmo que tenha ganho essa permissão via role ou via GRANT.
 - **DENY SELECT ON** dbo.empresa **TO** lucas;
 - O usuário lucas não poderá dar **select** na tabela empresa, mesmo que ele tenha recebido um GRANT anteriormente. O Deny sobreprõem o GRANT e as Roles.
 - **DENY SELECT (Nome, CPF) ON** dbo.empresa **TO** lucas;
 - o usuário lucas não pode dar **SELECT somente** nas colunas nome e cpf da tabela dbo.Empresa

BD II: Segurança

- **Você ainda não tem acesso** a nenhum objeto do banco!!! Para ter acesso, você deve definir as permissões da conta, usando **GRANT**, **DENY** e **REVOKE**.
- **REVOKE** : Remove uma permissão que foi dada explicitamente.
 - `REVOKE SELECT ON dbo.empresa TO lucas;`
 - O usuário lucas tem sua permissão reiniciada, ou seja, volta as permissões originais
 - REVOKE = tira o GRANT, ou mesmo um DENY (volta ao padrão).
 - DENY = proíbe, mesmo que venha por herança.
 - O **REVOKE não proíbe**. Ele apenas **remove permissões dadas explicitamente**, sejam positivas (GRANT) ou negativas (DENY).

BD II: Segurança

- **Você ainda não tem acesso** a nenhum objeto do banco!!! Para ter acesso, você deve definir as permissões da conta, usando **GRANT**, **DENY** e **REVOKE**

- **GRANT SELECT** ON **Pagar** TO user_consulta ;
- **GRANT INSERT** ON PAGAR TO user_gravacao ;
- **GRANT SELECT** ON pagar (idpagar,fkempresa TO user_consulta
- **DENY SELECT** ON pagar (idpagar,fkempresa TO user_consulta
- **REVOKE SELECT** ON pagar (idpagar,fkempresa) TO user_consulta

- **GRANT CONTROL ON DATABASE::uvv** TO todos;
- **GRANT SELECT** ON SCHEMA::dbo TO usuario;
- **GRANT EXECUTE** TO usu_todos ; // qualquer SP
- **GRANT EXECUTE ON OBJECT::"SP_iu_pagar"** TO usu_todos ;

BD II: Segurança

- Caso você seja o DONO (OWNER) do banco, não é necessário conceder permissões com GRANT, pois você já tem todas as permissões
- Para visualizar o “dono” de cada banco use:

```
SELECT name, owner_sid AS OwnerLogin  
FROM sys.databases;
```

Tente usar o mesmo comando, usando **SUSER_SNAME()**.

```
SELECT name, SUSER_SNAME(owner_sid) AS OwnerLogin  
FROM sys.databases;
```

BD II: Segurança

- Pra visualizar as permissões de um usuário:

SELECT

```
    USER_NAME(grantee_principal_id) AS Usuario,  
    permission_name,  
    state_desc,  
    OBJECT_NAME(major_id) AS Objeto
```

FROM sys.database_permissions

```
WHERE grantee_principal_id = USER_ID('lucas');
```

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
- **ROLES:** é um conjunto de permissões que podem ser atribuídas a usuários ou logins. Roles permitem gerenciar permissões de forma centralizada, agrupando-as e simplificando a administração de segurança.
 - **Roles de Servidor:** Definem permissões no nível do servidor SQL, **aplicáveis a todos os bancos de dados**. Exemplos incluem sysadmin e serveradmin. **É atribuída a LOGINS.**
 - **Roles de Banco de Dados:** Definem permissões dentro de um banco de dados específico. Exemplos incluem db_datareader (permite leitura de dados), db_datawriter (permite escrita de dados) e db_owner (controle total do banco de dados). **É atribuída a USERS**

BD II: Segurança

- Criação de Contas: O sql server possui vários “níveis” de autenticação.
- **ROLES:** Funcionam como um GRUPO com determinados direitos, onde você pode acionar usuários

```
ALTER ROLE db_datawriter ADD MEMBER usu_todos;
```

```
ALTER ROLE db_datareader ADD MEMBER usu_consulta;
```

Role	Permissões principais	Ideal para
db_owner	Controle total dentro do banco (criar, alterar, excluir objetos, gerenciar permissões).	Administradores do banco
db_securityadmin	Pode gerenciar usuários, roles e permissões dentro do banco.	Gerentes de segurança local
db_accessadmin	Pode adicionar/remover usuários e logins no banco.	DBAs locais
db_backupoperator	Pode realizar backups do banco.	Operadores de backup
db_ddladmin	Pode criar, alterar e excluir objetos (<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code>).	Desenvolvedores
db_datareader	Pode ler (<code>SELECT</code>) todas as tabelas e views.	Analistas, BI, relatórios
db_datawriter	Pode inserir, atualizar e deletar (<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>) em todas as tabelas.	Usuários operacionais
db_denydatareader	Explicitamente nega SELECT em tudo.	Segurança (restrições)
db_denydatawriter	Explicitamente nega INSERT/UPDATE/DELETE em tudo.	Segurança (restrições)
10 public	Permissões básicas (todo usuário pertence a ela por padrão).	Base mínima de acesso

Principais Server Roles

Role	Descrição	Nível de acesso	Recomendada para	
sysadmin	Controle total sobre tudo (bancos, logins, configurações, jobs, etc.).	Total (root/admin)	DBAs principais	
serveradmin	Pode alterar configurações de servidor (<code>sp_configure</code>), gerenciar opções globais e reiniciar o serviço.	Alto	Administradores de instância	
securityadmin	Pode criar, alterar e excluir logins, gerenciar permissões de servidor e roles.	Alto	Administradores de segurança	
processadmin	Pode finalizar sessões/processos (<code>KILL</code>) no servidor.	Moderado	Suporte técnico / monitoramento	
setupadmin	Pode gerenciar linked servers e configurações de setup.	Moderado	DBAs avançados	
diskadmin	Pode gerenciar arquivos físicos (backup devices, paths, etc.).	Moderado	DBA de infraestrutura	
dbcreator	Pode criar, alterar e apagar bancos de dados.	Limitado	Desenvolvedores / automação	
bulkadmin	Pode executar operações BULK INSERT (importação em massa).	Limitado	Usuários de ETL / integração	

BD II: Segurança

Tipo	Role	Descrição
Servidor	sysadmin	Controle total sobre o servidor SQL.
	serveradmin	Gerencia configurações de servidor.
	securityadmin	Gerencia logins e permissões de segurança.
	dbcreator	Cria, altera, restaura e remove bancos de dados.
Banco de Dados	db_owner	Controle total sobre o banco de dados.
	db_datareader	Permite leitura em todas as tabelas e views.
	db_datawriter	Permite escrita (inserir, atualizar, excluir) em todas as tabelas e views.
	db_backupoperator	Executa backups de banco de dados.
	db_ddladmin	Executa comandos de definição de dados (DDL) no banco de dados.
	db_denydatareader	Nega permissão de leitura em todas as tabelas e views.
	db_denydatawriter	Nega permissão de escrita em todas as tabelas e views.

BD II: Segurança

- **ROLES:** É possível criar ROLES personalizáveis

```
USE uvw
```

```
CREATE ROLE Role_financeira;
```

```
Go
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON pagar TO Role_financeira;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON receber TO Role_financeira;
```

--inserir usuário na roles

```
ALTER Role_financeira ADD usu_financeiro
```

BD II: Segurança

■ SELECT

```
    r.name          AS RoleName,  
    r.type_desc     AS RoleType,  
    m.name          AS MemberName,  
    m.type_desc     AS MemberType  
FROM sys.database_role_members drm  
INNER JOIN sys.database_principals r  
    ON drm.role_principal_id = r.principal_id  
INNER JOIN sys.database_principals m  
    ON drm.member_principal_id = m.principal_id  
ORDER BY r.name, m.name;
```